

รายงาน Mini Project

เรื่อง: การพัฒนาเว็บไซต์ **Full-Stack** ระบบบันทึกหนังที่ดู (**Movie Log**) ด้วย **REST API**

1. บทนำและวัตถุประสงค์ (Introduction & Objectives)

โครงการนี้จัดทำขึ้นเพื่อฝึกฝนทักษะการพัฒนาเว็บไซต์แบบ Full-Stack Application โดยสร้าง REST API ฝั่งเซิร์ฟเวอร์ด้วย Node.js และ Express.js เชื่อมต่อกับฝั่งไคลเอนต์ที่พัฒนาด้วย HTML, CSS และ JavaScript (Vanilla JS) ผ่าน fetch() API โดยระบบที่พัฒนาคือเว็บแอปสำหรับบันทึกรายการหนังที่ดูแล้ว สามารถเพิ่ม แก้ไข ลบ ค้นหา และดูคะแนนเฉลี่ยของหนังได้ ข้อมูลถูกเก็บในไฟล์ JSON

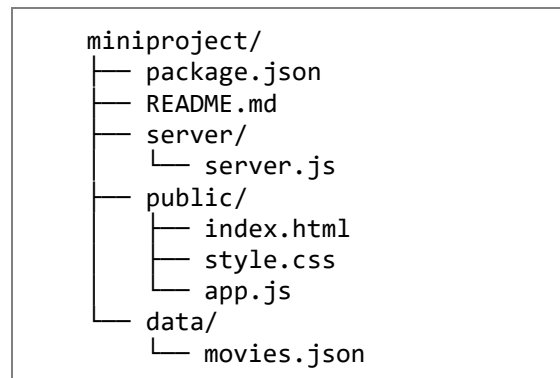
วัตถุประสงค์หลัก

- เพื่อออกแบบและพัฒนา REST API ที่รองรับการทำงานพื้นฐานตามหลัก CRUD (Create, Read, Update, Delete) ได้อย่างครบถ้วน
- เพื่อสร้างการเชื่อมต่อระหว่างฝั่งไคลเอนต์และฝั่งเซิร์ฟเวอร์โดยไม่มีการโหลดหน้าเว็บใหม่ (Asynchronous Operation / Single Page Application Concept)
- เพื่อฝึกจัดโครงสร้างโปรเจกต์ให้เป็นระเบียบ โดยแยกส่วนเซิร์ฟเวอร์ (/server) ส่วนไคลเอนต์ (/public) และส่วนข้อมูล (/data) ชัดเจน
- เพื่อศึกษาและประยุกต์ใช้ HTTP Status Codes และ Response Messages อย่างเหมาะสมตามมาตรฐาน RESTful

2. โครงสร้างโปรเจกต์และสถาปัตยกรรมระบบ (Project Architecture & Structure)

แอปพลิเคชันถูกจัดโครงสร้างไฟล์เพื่อแยกความรับผิดชอบของเซิร์ฟเวอร์ ไคลเอนต์

และข้อมูลให้อยู่อย่างเป็นสัดส่วน ดังรูปที่ 1



รูปที่ 1 แสดงสถาปัตยกรรมระบบ

- **Backend (/server/server.js):** ทำหน้าที่เป็น REST API Server

สำหรับรับส่งข้อมูลรูปแบบ JSON จัดการตรรกะของระบบ Validation

ตรวจสอบข้อมูลก่อนบันทึก และจัดการ Error Response รวมถึงให้บริการไฟล์ Static

ในโฟลเดอร์ /public

- **Frontend (/public):** ทำหน้าที่แสดงผล UI เชื่อมต่อ API ด้วย fetch()

และอัปเดตข้อมูลบนหน้าจอทันทีโดยไม่มีเฟรชเบราว์เซอร์

- **Data (/data/movies.json):** ไฟล์ JSON ที่ใช้เก็บข้อมูลหนังแทนฐานข้อมูล

เซิร์ฟเวอร์จะสร้างไฟล์นี้ให้อัตโนมัติ (ค่าเริ่มต้นเป็น []) หากยังไม่มี

3. การออกแบบ REST API (REST API Specification)

ข้อมูลหนังในระบบ (Data Model) ประกอบด้วยฟิลด์สำคัญ ดังนี้: id (String), title (String), year (Number), rating (Number: จำนวนเต็ม 0–5), watchedOn (String: วันที่ดู) และ notes (String)

HTTP Method	Endpoint	Request Body	Response Status
GET	/api/movies	-	200 OK
GET	/api/movies/:id	Route Parameter: :id	200 OK 404 Not Found
POST	/api/movies	JSON Body: { title, year, rating, watchedOn, notes }	201 Created 400 Bad Request
PATCH	/api/movies/:id	Route Parameter: :id JSON Body: ฟิลด์ที่แก้ไข	200 OK 400 Bad Request 404 Not Found
DELETE	/api/movies/:id	Route Parameter: :id	204 No Content 404 Not Found

HTTP Method	Endpoint	คำอธิบายการทำงาน
GET	/api/movies	ดึงรายการหนังทั้งหมดที่บันทึกไว้ในไฟล์ data/movies.json
GET	/api/movies/:id	ดึงข้อมูลหนังรายการเดียวตาม ID ตอบกลับ 404 หากไม่พบ
POST	/api/movies	เพิ่มหนังเรื่องใหม่ ตรวจสอบว่ามีชื่อเรื่องและคะแนนเป็นจำนวนเต็ม 0–5 ก่อนบันทึก (ตอบกลับ 400 หากข้อมูลไม่ถูกต้อง)
PATCH	/api/movies/:id	แก้ไขข้อมูลหนังเฉพาะฟิลด์ที่ส่งมา (title, year, rating, watchedOn, notes) ตอบกลับ 400 หากข้อมูลไม่ถูกต้อง และ 404 หากไม่พบ ID
DELETE	/api/movies/:id	ลบรายการหนังออกจากระบบ ตอบกลับสถานะ 204 เมื่อสำเร็จ และ 404 หากไม่พบ ID

4. การทำงานฝั่งไคลเอนต์ (Frontend Implementation)

ฝั่งไคลเอนต์ได้รับการออกแบบและพัฒนาให้สอดคล้องกับข้อกำหนด ดังนี้:

- การแสดงผลข้อมูล (Read / GET):** ใช้ฟังก์ชัน `fetch('/api/movies')`
เพื่อนำรายการหนังมาแสดงผลเป็นการ์ดรายการบนหน้าเว็บ
พร้อมแสดงจำนวนหนังและคะแนนเฉลี่ยด้านบน
- การค้นหาข้อมูล (Search):** เมื่อผู้ใช้พิมพ์ในช่องค้นหา
ระบบจะกรองรายการจากชื่อเรื่องและหมายเหตุที่โหลดมาแล้วบนฝั่งไคลเอนต์
แล้วแสดงผลทันทีโดยไม่ต้องเรียก API ซ้ำ
- การเพิ่มรายการใหม่ (Create / POST):** เมื่อส่งฟอร์ม ระบบจะเรียก `fetch()` ด้วย
HTTP Method POST พร้อมส่ง JSON Body ไปยังเซิร์ฟเวอร์
แล้วอัปเดตรายการหนังบน UI ทันทีโดยไม่ต้องโหลดหน้าเว็บใหม่
- การแก้ไขรายการ (Update / PATCH):** กดปุ่ม "แก้ไข" เพื่อดึงข้อมูลเข้าสู่ฟอร์ม
(ปุ่มเปลี่ยนเป็น "บันทึกการแก้ไข") เมื่อแก้ไขแล้วจะส่งคำขอ `PATCH /api/movies/:id`
ไปยัง API และสามารถกด "ยกเลิก" เพ้อออกจากโหมดแก้ไขได้
- การลบรายการ (Delete / DELETE):** กดปุ่ม "ลบ" ระบบจะแสดงกล่องยืนยันก่อน
จากนั้นส่งคำขอ `DELETE /api/movies/:id` เมื่อเซิร์ฟเวอร์ตอบกลับ `204 No Content` ไคลเอนต์จะลบการ์ดเล่มนั้นออกจาก UI ทันที
- การจัดการข้อผิดพลาด (Error Handling):** ครอบการเรียก `fetch()` ด้วย
`try/catch` และแสดงข้อความแจ้งผู้ใช้เมื่อโหลด บันทึก หรือลบข้อมูลไม่สำเร็จ